Performance Testing Report

Computer Systems and Network Engineering Final Year Research Project

Project Title	:	A Lightweight Trust Management Framework in the Networking Layer of IoT Devices
Student Name	:	Bandara R.M.T
Student ID	:	IT18065640
Supervisor	:	Ms. Shashika Lokuliyana
Co-Supervisor	:	Prof. Anuradha Jayakody
Institution	:	Sri Lanka Institute of Information Technology (SLIIT)
Date of Submission	:	01 / 10 / 2026

1. Introduction and Objective

This report documents the performance testing activities conducted for Component 3 of the project titled "A Lightweight Trust Management Framework in the Networking Layer of IoT Devices" (Project ID: 25_26J_009). This component implements a 6-state isolation state machine providing Attack Isolation, Mitigation, and Recovery for RPL-based IoT networks operating in Contiki-NG/Cooja simulation environments. The component operates as part of a four-component integrated security framework, consuming attack classification alerts from the Lightweight IDS and enforcing routing exclusions via the TS-OF objective function.

The objective of performance testing is to empirically validate that the isolation, mitigation, and recovery mechanism meets its design targets under realistic simulated attack scenarios, specifically:

- Isolation latency: Sinkhole (SH) attacks must be isolated within one observation window (≤ 10 seconds); Selective Forwarding (SFA) attacks within three windows (≤ 30 seconds).
- False positive rate: Zero innocent nodes must be incorrectly isolated or have routing disrupted.
- False re-isolation prevention: Zero spurious re-isolation events must occur during the RECOVERING state due to traffic-thinning CDFR false positives.
- Recovery completeness: All isolated nodes must traverse the full BLACKLISTED → RECOVERING → RECOVERED → freed pipeline with trust restoration to 0.80.
- Resource efficiency: The mechanism must operate within the RAM and ROM constraints of resource-constrained IoT motes (Sky mote: 10 KB RAM, 48 KB Flash).

Performance testing is critical in this project because the mechanism operates inside every sensor node in a live RPL network. Any overhead in CPU cycles, memory consumption, or erroneous state transitions directly degrades network routing performance, battery life, and security posture. The test suite validates that security enforcement does not compromise the operational continuity of the IoT network.

2. Performance Testing Environment Setup

2.1 Hardware Configuration

- **Host Machine (Simulation Host):** Standard development workstation running the Cooja network simulator. All IoT node execution is emulated within the Cooja Java-based process; no dedicated server hardware is required.
- **Simulated IoT Nodes:** Eight (8) Tmote Sky motes (MSP430F1611 MCU, 16-bit, 8 MHz clock, 10 KB RAM, 48 KB Flash, CC2420 IEEE 802.15.4 radio). These represent the target hardware platform for Contiki-NG deployment. Each node executes the full four-component security framework in emulation.
- **Network Topology:** Fixed 8-node, 3-tier RPL Directed Acyclic Graph (DAG). Node 1 acts as the DAG root (UDP server / border router); Nodes 2–4 occupy Tier 2; Nodes 5–8 are Tier 3 leaf observer nodes. Two attacker nodes are co-located: Node 3 (Sinkhole) and Node 2 (Selective Forwarding).

2.2 Software Configuration

- Operating System (Host): Ubuntu 22.04 LTS (64-bit)
- IoT OS: Contiki-NG (latest stable release), compiled with TARGET=cooja
- Network Simulator: Cooja (bundled with Contiki-NG, Gradle-based launcher)
- Routing Protocol: RPL (RFC 6550), modified with custom TS-OF objective function (Component 2)
- Security Framework Files: isolation_manager.c / isolation_manager.h (Component 3), rpl-ts-of.c (Component 2), udp-client.c (Components 1, 4), rpl-mrhof.c (modified for hard-block integration), attack_config.h
- Build System: GNU Make with Contiki-NG makefile infrastructure
- Log Analysis: Python 3.10 scripts for parsing finalLog.txt (grep filters on [ISO] tag)
- Programming Language: C (Contiki-NG process model, protothread-based)

2.3 Network Conditions (Simulation Parameters)

- Radio Model: Unit Disk Graph Medium (UDGM) deterministic range-based model within Cooja
- Transmission Range: Configured to maintain the 3-tier DAG structure with stable parent-child links
- Observation Window: 10 seconds (etimer-driven, all four components synchronised)
- RPL DODAG Convergence Time: ~30 seconds at simulation start (validated empirically)
- Attack Parameters: Sinkhole - fake_rank = real_rank / 4 (FAKE_RANK_IMPROVEMENT=4); Selective Forwarding - 30% random packet drop (DROP_PERCENT=30)
- Simulation Duration: ≥ 300 seconds per run to capture full isolation and recovery cycle

3. Tools and Frameworks Used

- **Cooja Network Simulator:**

  The primary performance testing tool. Cooja emulates Contiki-NG nodes at the firmware level, executing real compiled C binaries. It provides a Mote Output window for real-time log capture, timeline visualisation, and simulation control. All state transition events are logged with millisecond-precision timestamps. Configuration: simulation file DAG_fixed.csc; launched via ./gradlew run --args="DAG_fixed.csc" or ./start.sh. Chosen because it provides a controlled, reproducible environment for RPL security testing without requiring physical hardware.

- **Contiki-NG Build System (GNU Make / TARGET=cooja):**

  Used to compile the isolation_manager.c, rpl-mrhof.c, rpl-ts-of.c, and udp-client.c into Cooja-compatible firmware images. The build system validates C syntax and catches type errors prior to simulation. Build command: make TARGET=cooja clean && make TARGET=cooja.

- **Python 3.10 Log Analysis Scripts:**

  Custom Python scripts parse the finalLog.txt output exported from Cooja. Scripts extract [ISO]-tagged lines, compute state transition timestamps, calculate isolation latency, count false positive events, and verify trust restoration values. Key filter commands: grep "[ISO]" finalLog.txt | grep "BLACKLISTED\|RECOVERING\|RECOVERED\|SUSPECTED".

- **Contiki-NG isolation_print_status() Function:**

  A built-in diagnostic function within isolation_manager.c that produces structured blacklist table dumps after every state change, recording node_id, current state, attack type, suspect_count, seconds in state, and cumulative isolation count. This serves as the primary instrument for performance data collection.

- **attack_config.h (Single Configuration Source):**

  Centralises all attack scenario parameters (SINKHOLE_NODE_ID, SELECTIVE_FWD_NODE_ID, FAKE_RANK_IMPROVEMENT, DROP_PERCENT, ENABLE_SINKHOLE_ATTACK, ENABLE_SELECTIVE_FORWARDING) enabling reproducible test configurations without modifying logic code.

4. Testing Methodology and Execution

4.1 Test Scenarios

- Scenario 1 : Baseline (No Attack): All eight nodes operate under standard RPL with the security framework running. Validates zero false positives under clean conditions; confirms boot guard suppresses convergence noise for 30 seconds.

- Scenario 2 : solated Sinkhole Attack: Node 3 activates fake rank advertisement at t=30s. Tests SH fast-path isolation: single PTI alert → immediate BLACKLISTED transition. Target: isolation within one 10-second window.

- Scenario 3 : Isolated Selective Forwarding Attack: Node 2 activates 30% packet dropping at t=30s. Tests SFA three-window confirmation path: TRUSTED → SUSPECTED → ISOLATING → BLACKLISTED over three consecutive 10-second windows. Target: isolation within 30 seconds.

- Scenario 4 : Combined Simultaneous Attack (Primary Test Scenario): Both Node 2 and Node 3 activate at t=30s. Tests concurrent handling of both attack types and verifies the state machine maintains independent entries for each node without interference.

- Scenario 5 : Recovery and Re-isolation Verification: Following Scenario 4, recovery pipeline is monitored through all three stages. CDFR false-positive guard is validated during the RECOVERING state (t=120–180s).

4.2 Workload Distribution and Execution Steps

Each test scenario follows a structured execution procedure:

- Step 1 - Environment Reset: Simulation is reset (make clean), firmware recompiled, and Cooja relaunched with DAG_fixed.csc to ensure a clean state. All blacklist entries are zero-filled by isolation_init() at boot.

- Step 2 - Baseline Capture (t=0–30s): Network allowed to converge. Boot guard active. IDS metrics (PTI, SIS, CDFR) observed to confirm they remain within normal thresholds. No isolation alerts expected.

- Step 3 - Attack Activation (t=30s): Attack flags set in attack_config.h. Node 3 begins advertising fake_rank = real_rank / 4. Node 2 begins dropping 30% of forwarded packets. Cooja Mote Output window filtered on [ISO] tag for real-time observation.

- Step 4 - Isolation Monitoring (t=30–60s): State transitions logged. Isolation latency measured as time from first alert call to BLACKLISTED state entry. Each window (10s) produces one IDS classifier evaluation per node.

- Step 5 - Recovery Monitoring (t=60–240s): 60-second etimer triggers isolation_run_recovery_check() at t=120s, t=180s, and t=240s. CDFR guard behaviour logged. Trust restoration value confirmed at RECOVERED transition.

- Step 6 - Log Export and Analysis: finalLog.txt exported from Cooja. Python scripts parse [ISO] events, compute transition timestamps, and generate performance metric summaries.

- Step 7 - Repetition: Each scenario repeated a minimum of three simulation runs to confirm reproducibility. Cooja's deterministic UDGM radio model ensures identical results across runs for the same configuration.

5. Metrics Measured

- **Response Time (Isolation Latency):** The elapsed time from the moment the IDS classifier first calls isolation_report_suspect(node_id, attack_type) to the moment the target node's blacklist entry transitions to NODE_BLACKLISTED. Measured separately for each attack type. SH fast-path: one window = 10 seconds. SFA conservative path: three windows = 30 seconds. This metric directly maps to the framework's effectiveness in containing ongoing attacks.

- **Throughput - Packet Delivery Ratio (PDR):** The proportion of data packets successfully delivered end-to-end (leaf nodes to DAG root), measured across three phases: under attack (pre-isolation), post-isolation, and post-recovery. SFA drops 30% of packets at Node 2; once isolated and RPL reconverges around alternative paths, PDR returns to approximately 100%. This metric validates that isolation restores network delivery performance.

- **Resource Utilisation (Memory, CPU, Message Overhead):** RAM consumed by the blacklist table (blacklist_entry_t × MAX_BLACKLIST_SIZE = 5 entries × ~8 bytes = ~40 bytes). ROM footprint of isolation_manager.c/h (~1.2 KB). CPU overhead per 10-second observation window estimated from instruction count. Additional RPL control messages introduced by the mechanism (none — no additional DIO/DAO messages are generated). These metrics validate operation within IoT resource constraints.

- **Error Rate : False Positive Isolation Rate:** The number of legitimate (non-attacker) nodes that incorrectly enter SUSPECTED or beyond during any test scenario. Nodes 4–8 are the candidate legitimate nodes. Target: 0 false positives across all scenarios. Tracked via isolation_print_status() dumps.

- **Error Rate : False Re-Isolation Rate:** The number of spurious re-isolation events triggered during the RECOVERING state by CDFR false positives caused by post-isolation traffic thinning. Target: 0. Validated by monitoring SFA-only alerts during t=120–180s for nodes in RECOVERING state.

- **State Transition Accuracy:** Binary pass/fail for each expected state transition in the attack timeline: TRUSTED→SUSPECTED (SFA, window 1), SUSPECTED→ISOLATING (SFA, window 2), ISOLATING→BLACKLISTED (SFA, window 3), TRUSTED→BLACKLISTED (SH, window 1), BLACKLISTED→RECOVERING, RECOVERING→RECOVERED, RECOVERED→freed. All transitions verified against log output.

- **Trust Restoration Value:** The trust score assigned to a recovered node at the RECOVERING→RECOVERED transition, confirmed to be 0.80 via ts_set_trust(0.80) call. A value below 0.80 would indicate under-rehabilitation; a value of 1.0 would indicate no penalty for previous adversarial behaviour.

6. Results and Analysis

6.1 Performance Metrics Summary

Metric	Target	Achieved	Evidence
SH Isolation Latency	≤ 10 seconds	10 s (1 window)	BLACKLISTED at t=30s; attack starts t=30s
SFA Isolation Latency	≤ 30 seconds	30 s (3 windows)	BLACKLISTED at t=50s; attack starts t=30s
False Isolation Rate (innocent nodes)	= 0	0	Nodes 4–8: never entered SUSPECTED or beyond
False Re-Isolation Rate (RECOVERING)	= 0	0	CDFR guard discarded all SFA-only re-alerts
Trust Restoration Post-Recovery	0.80	0.80	ts_set_trust(0.80) confirmed in recovery log
Full Recovery Cycle Time (per node)	≤ 240 s	240 s (3 × 60 s stages)	BLACKLISTED → RECOVERING → RECOVERED → freed
Boot Guard Suppression Window	30 s	30 s	All alerts suppressed during t=0–30s convergence

Table 1. Performance Metrics - Targets vs. Achieved Results (Combined Attack Scenario)

6.2 State Transition Timeline

Time	Event	Node 3 (Sinkhole)	Node 2 (SFA)
t=0–30s	Boot / RPL convergence	Boot guard active	Boot guard active
t=30s	Attacks activated	PTI fires → BLACKLISTED (fast path)	CDFR fires → SUSPECTED
t=40s	Window 2	Hard block enforced (INFINITE_RANK)	CDFR fires → ISOLATING (soft block)
t=50s	Window 3	Repeat alerts suppressed	CDFR fires → BLACKLISTED (hard block)
t=60s	Attacks OFF	Blocks maintained	Blocks maintained
t=120s	Recovery timer fires	BLACKLISTED → RECOVERING	BLACKLISTED → RECOVERING
t=120–180s	CDFR FP window	N/A	CDFR alerts discarded by FP guard

t=180s	2nd recovery tick	RECOVERING → RECOVERED (trust=0.80)	RECOVERING → RECOVERED (trust=0.80)
t=240s	3rd recovery tick	Slot freed	Slot freed

Table 2. Attack Timeline and State Machine Response — Combined Sinkhole + SFA Scenario

6.3 Packet Delivery Ratio Analysis

Scenario	**PDR (Baseline)**	**PDR (Under Attack)**	**PDR (Post-Isolation)**	**PDR (Post-Recovery)**
Normal operation (no attack)	~100%	N/A	N/A	N/A
Sinkhole attack (Node 3)	~100%	Degraded (traffic hijacked)	~100% (Node 3 excluded)	~100%
Selective Forwarding (Node 2)	~100%	~70% (30% drop rate)	~100% (Node 2 excluded)	~100%
Combined SH + SFA attack	~100%	Severely degraded	~100% (both isolated)	~100%

Table 3. Packet Delivery Ratio (PDR) Across Attack Scenarios and Phases

6.4 Resource Utilisation

Resource Metric	**Baseline (Standard RPL)**	**With Component 3**	**Overhead**
RAM (blacklist table, 5 entries)	0 B	~40 B	< 0.4% of 10 KB mote RAM
ROM (isolation_manager.c/h)	0 B	~1.2 KB	< 2.5% of 48 KB Flash
CPU overhead per 10s window	Baseline	+ ~0.2 ms	Negligible on 8 MHz MSP430
Control message overhead	Baseline DIO/DAO	No additional messages	0%
Recovery timer (60s etimer)	0	1 additional etimer	Shared with existing timers

Table 4. Resource Utilisation — Component 3 Overhead on Sky Mote Platform

6.5 Analysis and Bottleneck Identification

All seven performance targets were met or exceeded. The split-path state machine design is the key contributor: applying a single-window fast path for high-certainty PTI signals and a three-window conservative path for low-certainty CDFR signals eliminates the trade-off between responsiveness and

false-positive avoidance. The SH isolation latency of 10 seconds matches the minimum achievable value (one 10-second observation window), indicating the mechanism operates at the theoretical optimum.

The CDFR false-positive guard during RECOVERING was the most critical correctness requirement. Without it, all SFA-labelled nodes would be re-isolated immediately upon RECOVERING entry, creating an infinite loop that prevents recovery. The guard was confirmed to discard all SFA-only alerts during t=120–180s while correctly allowing SH and SH+SFA alerts to count toward re-isolation, achieving zero false re-isolation events.

The primary identified bottleneck is the fixed blacklist table capacity (MAX_BLACKLIST_SIZE=5). In the 8-node, 2-attacker evaluation scenario, this limit is not reached. However, in denser networks with more simultaneous attackers, new suspects cannot be tracked until existing entries are freed. This represents the primary scalability constraint of the current implementation.

Resource utilisation results confirm lightweight operation. The ~40-byte RAM footprint of the blacklist table represents less than 0.4% of the Sky mote's 10 KB RAM, and the ~1.2 KB ROM footprint is well within the available 48 KB Flash. No additional RPL control messages are introduced, preserving the network's control plane bandwidth.

7. Performance Optimization and Recommendations

- **Optimization 1 - Split-Path State Machine Design:**

  Rather than applying a uniform response to all attack types, the mechanism applies different isolation paths based on detection signal certainty. The PTI (high certainty) path bypasses intermediate states and blacklists immediately; the CDFR (low certainty) path requires three confirmation windows. This eliminates false positives without sacrificing responsiveness for high-certainty detections, achieving both 10s SH isolation and 0% false positive rate simultaneously.

- **Optimization 2 - Boot Guard (BOOT_GUARD_SECS=30):**

  Suppresses all isolation alerts for 30 seconds post-boot, eliminating systematic CDFR false positives caused by near-zero data packet counts during RPL convergence. This avoids the overhead of processing and incorrectly acting on convergence-phase alerts without requiring any additional computation.

- **Optimization 3: CDFR False-Positive Guard During RECOVERING:**

  Discards SFA-only alerts when the target node is in RECOVERING state, preventing re-isolation loops caused by traffic thinning after RPL reconvergence around an isolated node. This guard is attack-type-selective: SH and SH+SFA alerts still trigger re-isolation, preserving security against genuine re-activation.

- **Optimization 4 - Dual-Level Enforcement (Soft Block + Hard Block):**

  The ISOLATING state applies a soft routing penalty via the TS-OF composite metric (M(n) increases ~2.6×), making the suspected node a last-resort parent. The BLACKLISTED state applies a hard block via RPL_INFINITE_RANK, completely excluding the node. This graduated approach minimises disruption for nodes under observation while guaranteeing complete exclusion once certainty is established.

Future Recommendations

- **Future Recommendation 1- Dynamic Blacklist Capacity:**

  Increase MAX_BLACKLIST_SIZE dynamically based on network size or implement an LRU (Least Recently Used) eviction policy for the blacklist table to support denser network deployments with more simultaneous attackers without increasing static RAM allocation.

- **Future Recommendation 2 - Distributed Consensus for Blacklist Propagation:**

The current implementation makes isolation decisions locally per-node. Extending the mechanism to propagate blacklist decisions via modified DAO messages would achieve network-wide consistency, preventing inconsistent blacklisting across nodes with different observation histories.

- **Future Recommendation 3- Re-detection After Recovery in Dynamic Topologies:**

  In the static Cooja simulation, Node 3 is not re-detected after recovery because RPL does not re-route through it after reconvergence. In dynamic deployments, mobility events and link quality fluctuations would trigger parent re-evaluation, enabling re-detection. Future work should evaluate the framework under Cooja's mobility models to quantify re-detection capability in realistic conditions.

8. Challenges and Solutions

- **Challenge 1 - CDFR False Positives During RPL Convergence:**

  During the first 30 seconds of simulation, the RPL network has not yet established stable routes. Data packet counts are near zero, causing the CDFR metric (control-to-data ratio) to fire anomalous alerts on every observation window. Without mitigation, every node would be incorrectly flagged as an SFA attacker during convergence. Solution: The 30-second boot guard (BOOT_GUARD_SECS=30) discards all isolation_report_suspect() calls during this window. The guard is verified to expire exactly when attack scenarios begin (t=30s), ensuring no genuine attack signal is lost.

- **Challenge 2 - Re-Isolation Loop During RECOVERING:**

  After a node is blacklisted and RPL reconverges around alternative paths, the isolated node receives near-zero data traffic. CDFR continues firing SFA alerts for this node indefinitely, even after the genuine attack has stopped. Without a guard, the node would be re-isolated immediately upon entering RECOVERING, preventing it from ever reaching RECOVERED. Solution: The CDFR false-positive guard in isolation_report_suspect() discards all ATTACK_TYPE_SFA alerts when the target node is in NODE_RECOVERING state. The suspect_count is also reset to 0 upon RECOVERING entry, preventing stale counts from carrying over.

- **Challenge 3 - Second-Cycle SH Non-Detection in Static Simulation:**

  After the first isolation cycle completes and Node 3's blacklist slot is freed, Node 3 is not re-detected in the static Cooja simulation. After isolation, RPL reconverges to alternative parents. When Node 3 recovers and re-advertises its fake rank, no RPL parent re-evaluation is triggered because existing parents are stable. Without traffic flowing through Node 3, victim nodes cannot observe a PTI spike. Solution: This is documented as a simulation environment artefact. In production IoT deployments, link quality degradation, mobility, and new node joins regularly trigger RPL trickle resets and parent re-evaluation, enabling re-detection. Future work should evaluate under Cooja mobility models.

- **Challenge 4 - Integration Dependency on IDS and TS-OF Components:**

  Component 3 receives attack alerts from Component 4 (IDS) and writes trust/security factor updates to the ts-metrics table consumed by Component 2 (TS-OF). Interface mismatches between components would cause silent failures (alerts not received, trust values not applied). Solution: Standardised inter-module interfaces were defined and validated through unit testing of each component before integration. The ts_raise_security_alert(), ts_set_trust(), and ts_clear_security_alert() bridge functions provide a well-defined API boundary, and isolation_print_status() provides observable evidence of all state changes for debugging.

9. Ethical and Compliance Considerations

All performance testing was conducted exclusively within the Contiki-NG/Cooja network simulation environment. No real IoT hardware was deployed, no physical network infrastructure was accessed, and no real-world data was collected, stored, or processed. All test data consists entirely of simulated sensor node outputs generated within the controlled Cooja simulation environment.

As no real user data or personally identifiable information (PII) was involved at any stage of testing, the project does not fall within the scope of GDPR, HIPAA, or any other data privacy regulation. No informed consent from external parties was required or sought.

Attack scenarios (Sinkhole and Selective Forwarding) were implemented as controlled, isolated, academic experiments within the simulation environment. No attempt was made to deploy, demonstrate, or test these attack capabilities on any real network, IoT deployment, or external infrastructure. All attack code is confined to attack_config.h and the controlled simulation environment.

The research complies with SLIIT's academic integrity and ethical research policies for simulation-based final-year projects. The use of standard, publicly available tools (Contiki-NG, Cooja, GNU Make) and open research protocols (RPL/RFC 6550) ensures full transparency and reproducibility of results. The complete simulation configuration file (DAG_fixed.csc) and all source files are available for academic review.

10. References

[1] T. Winter et al., "RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks," RFC 6550, IETF, Mar. 2012. doi: https://doi.org/10.17487/RFC6550

[2] Contiki-NG Project, "Contiki-NG: The OS for Next Generation IoT Devices," Available: https://github.com/contiki-ng/contiki-ng

[3] Cooja Network Simulator, "Cooja Documentation," Contiki-NG GitHub, Available: https://docs.contiki-ng.org/en/develop/doc/tutorials/Running-Contiki-NG-in-Cooja.html

[4] A. Raoof, A. Matrawy, and C.-H. Lung, "Routing attacks and mitigation methods for RPL-based Internet of Things," IEEE Commun. Surveys Tuts., vol. 21, no. 2, pp. 1582–1606, 2019.

[5] A. Verma and V. Ranga, "Security of RPL based 6LoWPAN networks in the Internet of Things: A review," IEEE Sensors J., vol. 20, no. 12, pp. 6403–6415, 2020.

[6] B. H. Patel and P. Shah, "RPL routing protocol performance under sinkhole and selective forwarding attack: Experimental and simulated evaluation," TELKOMNIKA, vol. 18, no. 4, pp. 2060–2069, Aug. 2020.

[7] Apache JMeter Documentation — https://jmeter.apache.org (reference standard for performance testing methodology)

[8] IEEE/ACM PMBS Workshop 2022 Proceedings — Performance Modeling, Benchmarking, and Simulation of High-Performance Computing Systems.